# A3-PS036-TC161

PQC DNSSEC MTL Benchmark Using a Reference Implementation (PS-036)

**Akash Joshi**: akashjoshi2023@gmail.com
**Anitha Natarajan**: anithapriyanatarajan@gmail.com
**Indu Radhakrishnan (Faculty)**: induradhakrishnan@pes.edu

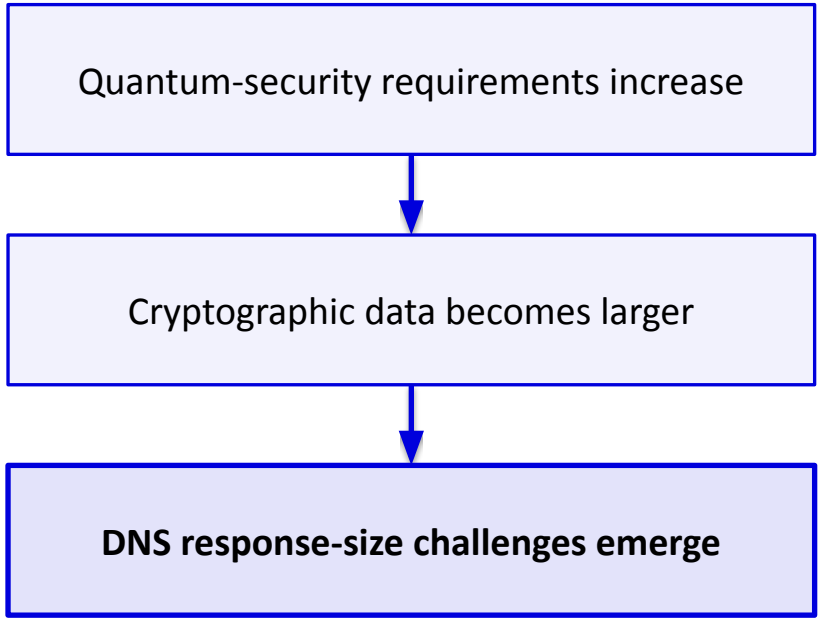
AIORI Hackathon 3

October 13, 2026

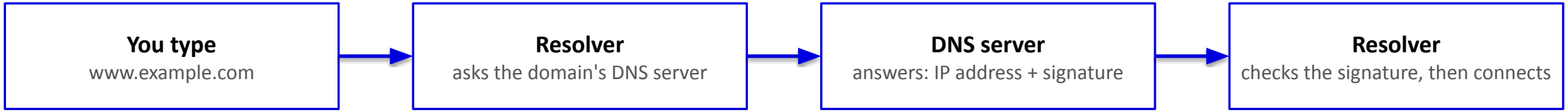
# The Problem: DNSSEC in the Quantum Era

**What is at risk?**

- **DNS** translates domain names into IP addresses so users can reach websites.
- **DNSSEC** uses digital signatures to verify DNS data and protect against tampering [1].
- **Quantum threat:** future quantum computers could break the classical cryptographic algorithms used by DNSSEC (RSA, ECDSA).
- **PQC** (Post-Quantum Cryptography) offers algorithms designed to resist quantum attacks, but some produce much larger keys and signatures [4].




## How one lookup works today

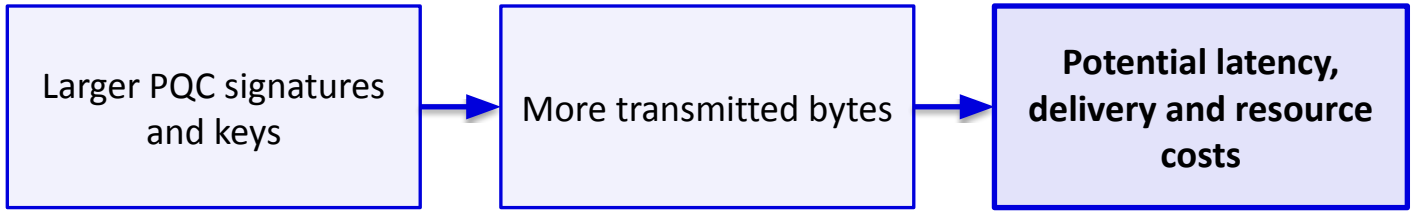



The signature travels inside every answer, so its size is paid on every lookup.

# The Technical Challenge

## Why does larger cryptographic data matter?

DNS commonly uses **UDP** for responses. Larger DNSSEC responses can exceed the permitted UDP message size (**1,232 bytes** [3, 11]), potentially causing **truncation**, **IP fragmentation**, **packet loss**, or **TCP fallback**.




A truncated answer must be asked for again over TCP: an extra round trip, more bytes and more work for every resolver.

**Signature size vs one 1,232-byte UDP packet**

| Signature type | Size | Fits? |
|---|---|---|
| ECDSA P-256 (today) | 64 B | yes |
| RSA-2048 (today) | 256 B | yes |
| **SLH-DSA-128s (PQC, we test)** | **7,856 B** | **no, 6× too big** |

### Core problem to investigate

**Can quantum-resistant DNSSEC authentication remain practical without excessive transmission and computational overhead?**

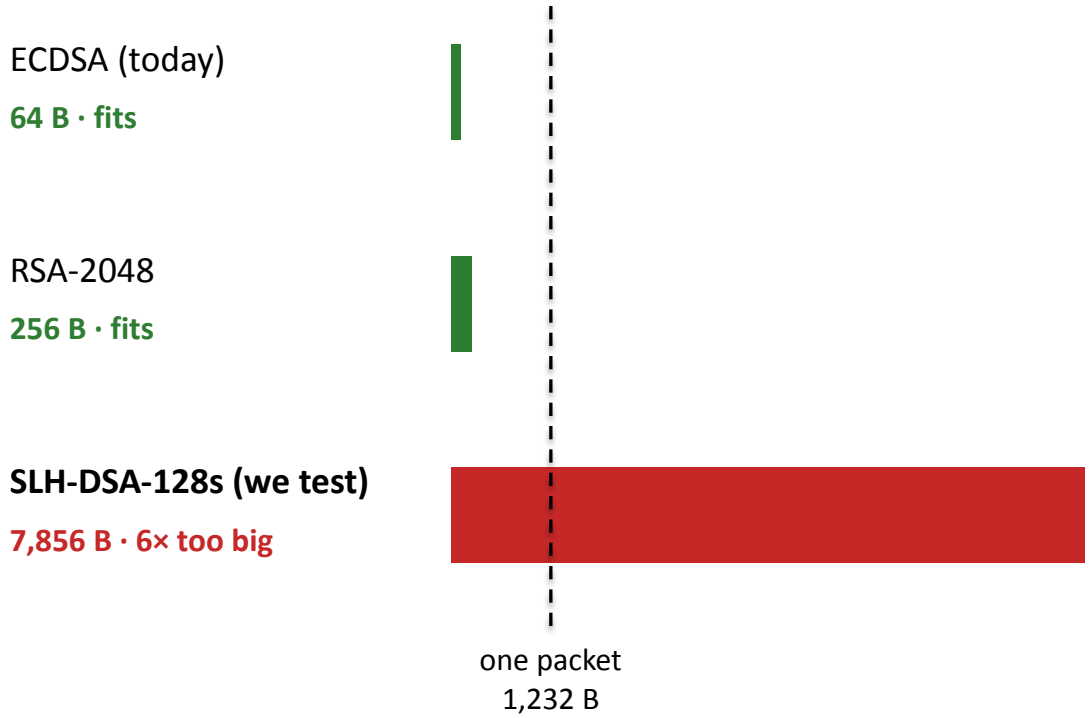
The trade-offs must be measured when comparing MTL and non-MTL workloads:

- **Signing time**
- **Verification time**
- **Transmitted bytes**
- **Cache usage**

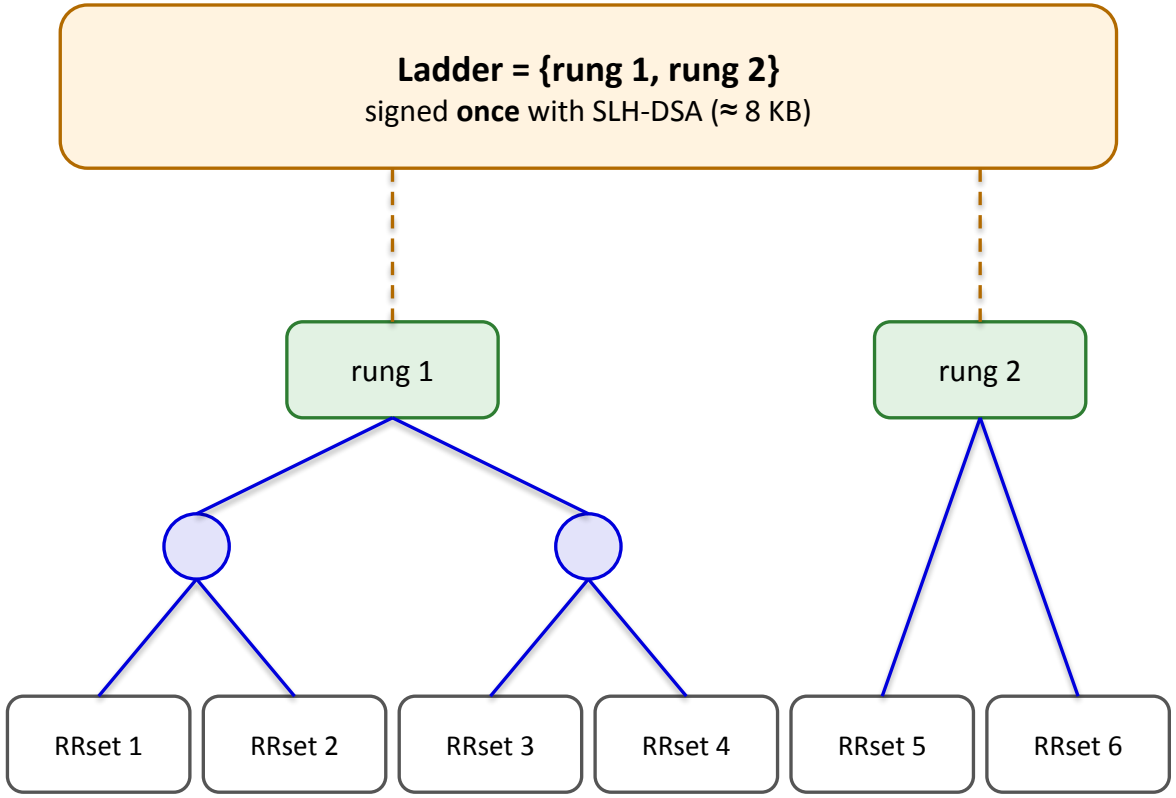
# Background (1/3): why size matters

- **DNSSEC** attaches a signature to each set of DNS records, so a resolver can check that an answer is genuine [1].
- Answers normally travel in **one small packet** (≤ 1,232 bytes) [3, 11].
- If an answer is bigger, it is **cut off**, and the resolver must ask again over a slower connection (TCP).
- **SLH-DSA** is the safest quantum-resistant option: it relies only on hash functions and has a tiny key. But its signature is **huge** [4].

**Signature size compared with one packet**



# Background (2/3): how MTL mode works

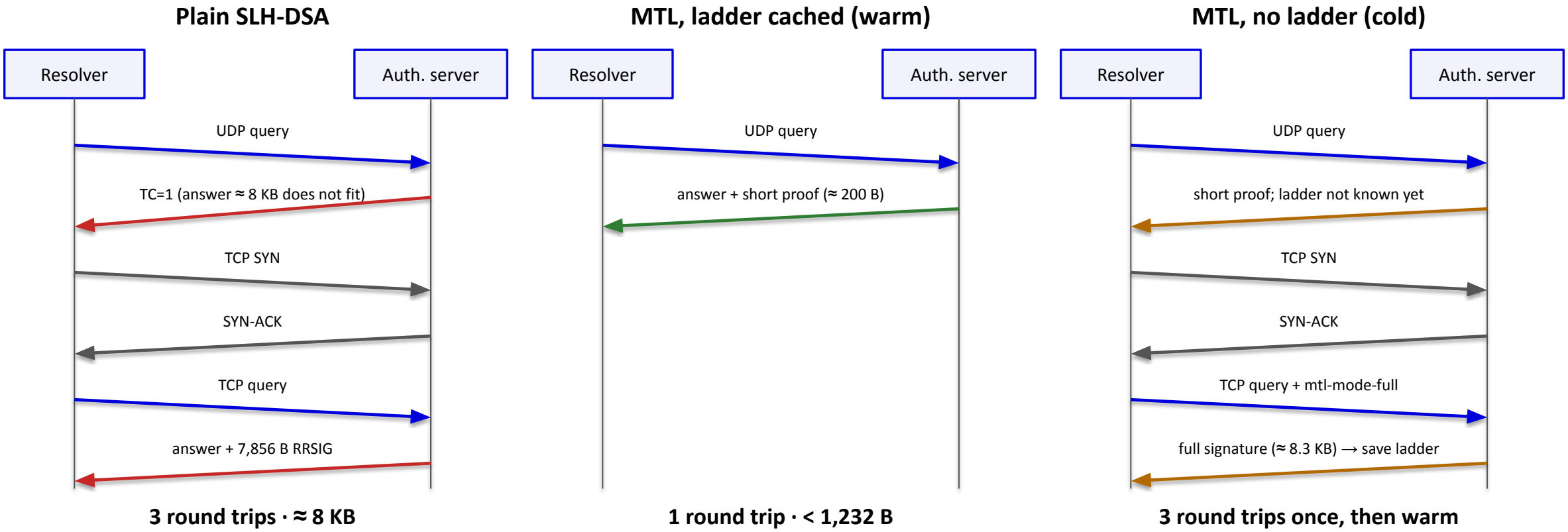



MTL mode is defined in [5]; its use in DNSSEC in [6].

**Analogy:** instead of a notary signing every page of a long contract, the notary signs the **table of contents** once. Each page carries a short note proving where it sits in that table.

| Term | What it means |
|---|---|
| **Ladder** | the signed “table of contents” (≈ 8 KB), signed **once** |
| **Short proof** | what each record carries: ≈ 200 B instead of 7,856 B |
| **Full signature** | short proof + ladder; only needed if the resolver hasn't saved the ladder yet |
| **Node hash** | the quick step linking a record to the ladder; far cheaper than a signature |

# Background (3/3): three ways a lookup can go



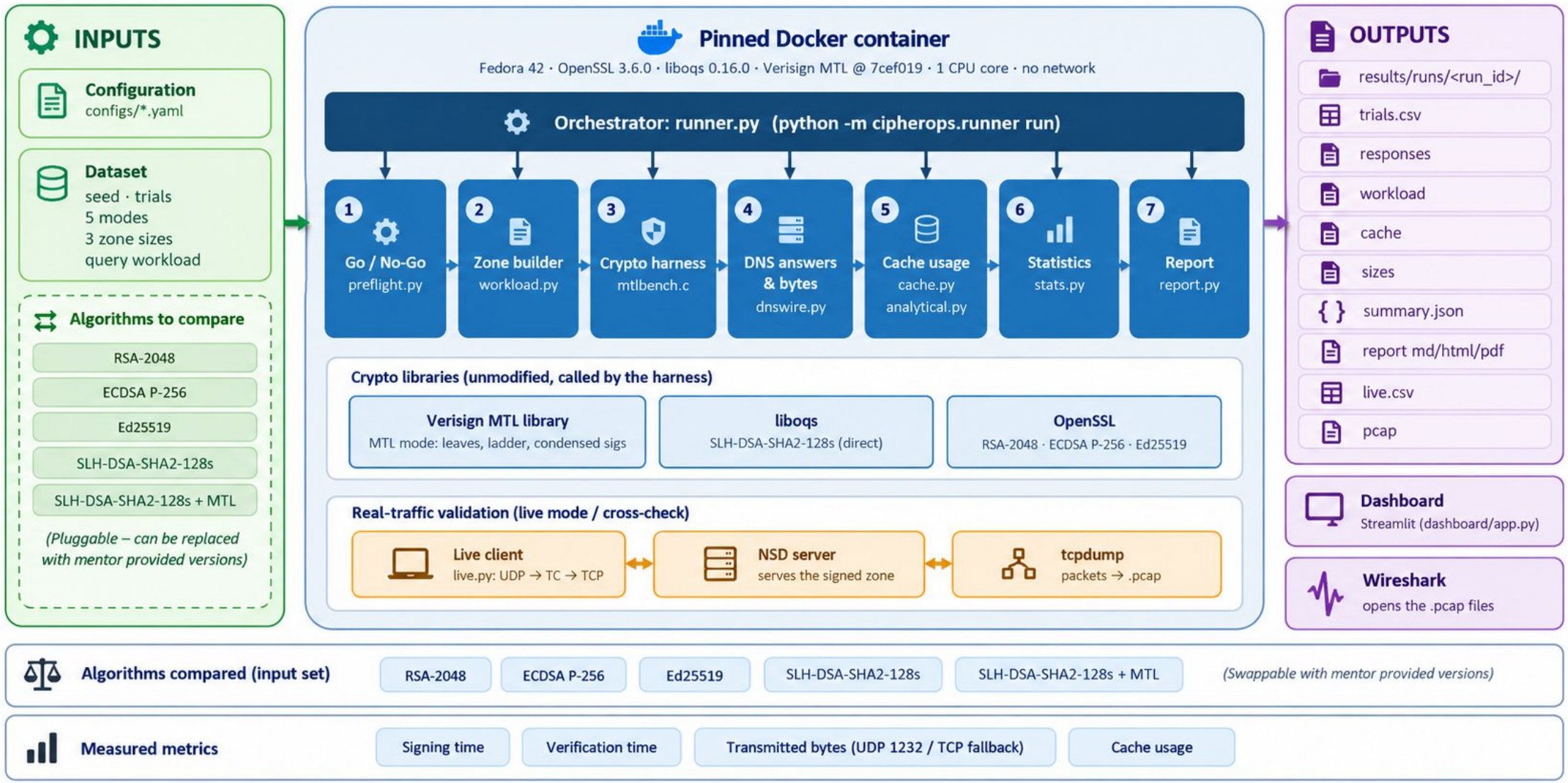

**Key point:** MTL's benefit depends on **how big the zone is** and **how often the resolver can reuse a saved ladder**. Our test is built around these two factors.

# Literature & RFC Review

**Only the sources that define what we build or measure.**

| Ref | Source | What it defines | Used in Cipherops for |
|---|---|---|---|
| [1] | RFC 4033 / 4034 / 4035: DNSSEC | RRSIG and DNSKEY formats; signing input (RFC 4034 §3.1.8.1) | Byte-exact signing input per RRset (workload.py) |
| [2] | RFC 5155 NSEC3; RFC 9276 | Hashed denial of existence; 0 iterations, no salt | NSEC3 chain in every zone; NXDOMAIN answers |
| [3] | RFC 6891 EDNS(0); RFC 7766 | UDP buffer size, TC bit, TCP fallback | 1,232 / 4,096-byte limits and TCP retries (dnswire.py) |
| [4] | NIST FIPS 205: SLH-DSA | The PQC signature scheme (SLH-DSA-SHA2-128s) | The algorithm under test, with and without MTL |
| [5] | draft-harvey-cfrg-mtl-mode-09 | MTL construction; §12 maximum sizes | Analytical size and cache estimates (analytical.py) |
| [6] | draft-fregly-dnsop-slh-dsa-mtl-dnssec-05 | MTL in RRSIG: MTL-Type byte, mtl-mode-full option | Encoding of MTL signatures in DNS answers |
| [7] | Fregly et al., CT-RSA 2023 | Design and size argument for MTL mode | The hypothesis this benchmark tests |
| [8] | Verisign MTL reference implementation | Supplied library, test suite and CLI tools | MTL signing and verification; go/no-go gate |
| [9] | liboqs 0.16.0; OpenSSL 3.6.0 | Direct SLH-DSA; RSA, ECDSA, Ed25519 | Non-MTL and classical baselines |
| [10] | RFC 5702 / 6605 / 8080 | DNSSEC algorithms 8, 13 and 15 | Classical reference modes |
| [11] | DNS Flag Day 2020 | 1,232-byte EDNS buffer recommendation | Default UDP limit |

# Proposed Solution



# Optimization Proposed

**One SLH-DSA signature per zone instead of one per RRset. Large zone (10,005 RRsets), medians of 5 trials:**

| Metric | SLH-DSA direct (before) | SLH-DSA + MTL (after) | Ed25519 (classical reference) |
|---|---|---|---|
| Signature in each RRSIG | 7,856 B | ≤ 284 B condensed<br>+ one 8,120 B signed ladder per zone | 64 B |
| Sign the whole zone | ≈ 92 min (extrapolated)[1] | **0.58 s** | 0.37 s |
| Verify one answer, ladder cached | 541 µs | **7.2 µs**[2] | 117 µs |
| Verify the first answer, nothing cached | 480 µs | 490 µs | 103 µs |
| Signature storage for the zone | 78.6 MB | **2.7 MB** (29× less) | 0.64 MB |
| Answers that fit one 1,232-byte UDP packet | 0% (median 7,982 B) | **87.4%** (median 411 B) | 91.5% (median 190 B) |
| Bytes on the wire, 762 lookups[3] | 11.08 MB; 762 TCP retries | **0.92 MB** (12× less); 246 TCP | 0.38 MB; 43 TCP |
| Time for 762 lookups, 50 ms round trip[3] | 125.6 s | **63.5 s** (2× faster) | 43.0 s |

[1] 1,000 of 10,005 RRsets signed per trial at 0.55 s each, scaled to the zone. Fully measured on the 1,005-RRset zone: 558 s direct vs 0.57 s with MTL. [2] Includes one ladder verification spread over the zone.

[3] Measured live: real client and NSD server, EDNS 1,232 B, 2,000-lookup workload with resolver cache. MTL: 245 truncated answers plus 1 ladder fetch.

# Solution Architecture & Design

**UML component and deployment view: two measurement paths, one stack profile, three unmodified crypto libraries.**

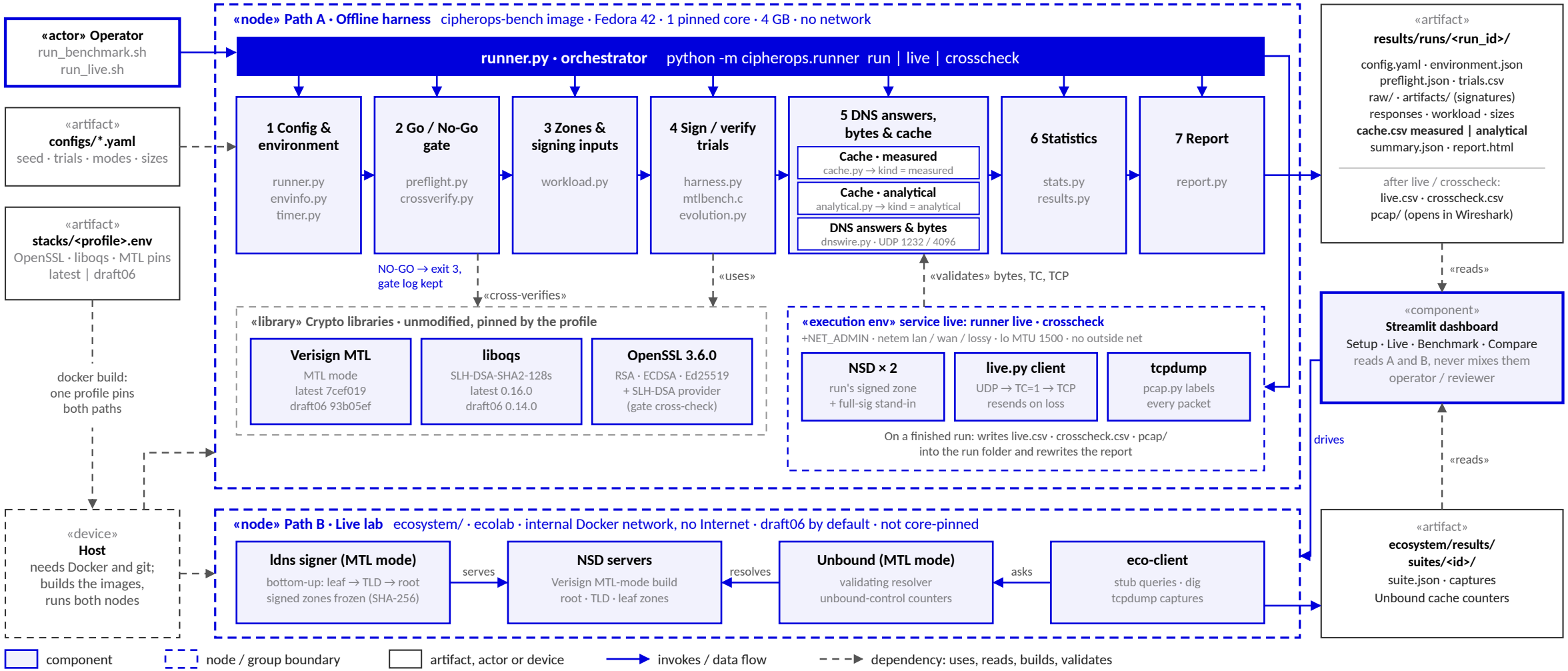
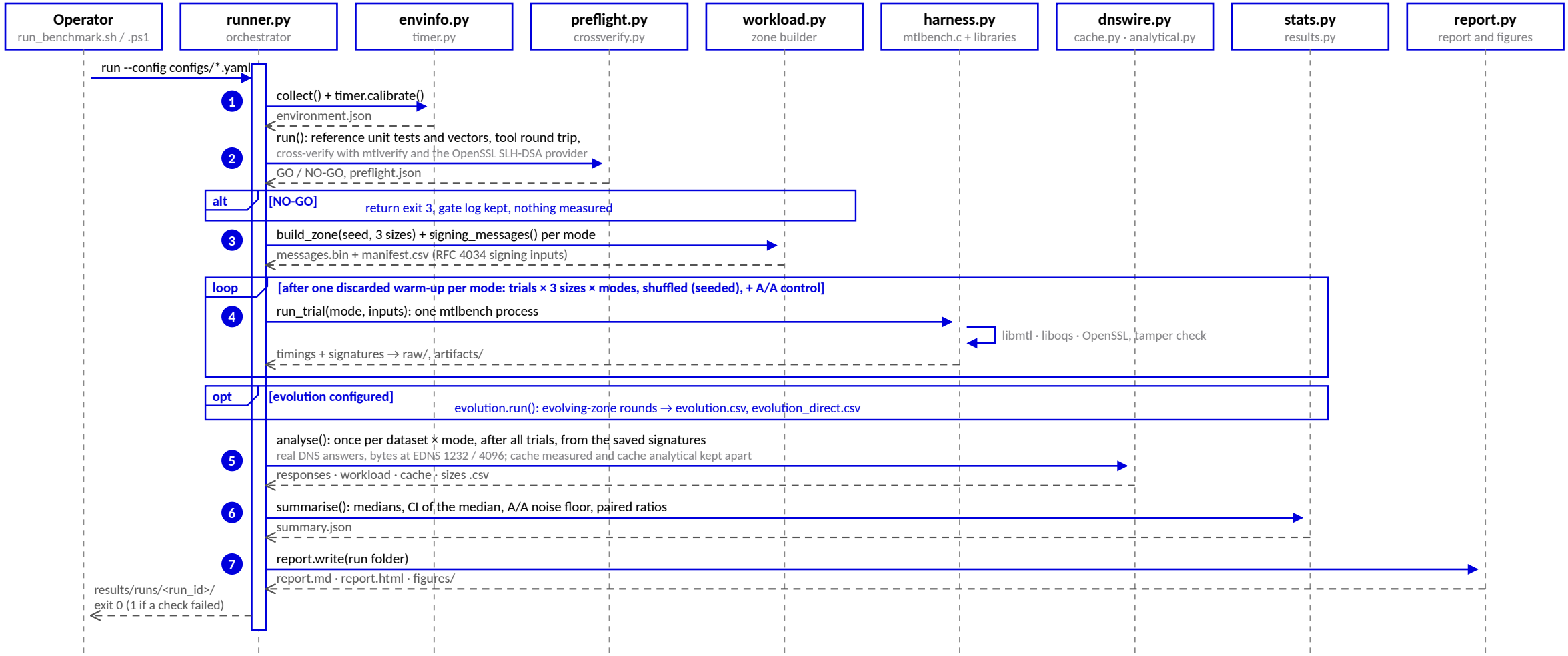



Figure: UML component and deployment view of Cipherops (branch phase1-demo-safety). Stage numbers match the runner's log.

# Runtime Design: One Benchmark Run

**UML sequence diagram of runner.py run: the same seven stages the runner prints, from configuration to results.**




*Later, on a finished run, runner live and runner crosscheck (compose service live) add live.csv, crosscheck.csv and pcap/, then rewrite the report.*

# Design Approach & Key Decisions

**Six design decisions behind the benchmark, and the trade-offs we accept.**

### 1 Reproducible by construction

One pinned Docker image (Fedora 42, OpenSSL 3.6.0, liboqs 0.16.0, Verisign MTL @ 7cef019) and one command run everything.

*Evidence: Host needs only Docker and git*

### 2 Fail fast with a Go / No-Go gate

Before any measurement, the supplied MTL library must pass its own tests and a tool round trip. A broken reference stops the run.

*Evidence: preflight.py; gate result stored with each run*

### 3 Reference libraries stay untouched

A thin C harness calls Verisign MTL, liboqs and OpenSSL as they are, so timings reflect the reference code and not ours.

*Evidence: libraries are called unmodified by mtlbench.c*

### 4 Identical input for every mode

Zones come from one seed. Each RRset's signing input is byte-exact per RFC 4034, so MTL and non-MTL sign the same data.

*Evidence: workload.py; seed fixed in the configuration*

### 5 A model, checked on real traffic

dnswire.py and cache.py model UDP limits, TC=1 and TCP retry. A real NSD server with packet capture cross-checks them.

*Evidence: 762 lookups measured on a real NSD server*

### 6 Every number is traceable

Each run keeps its configuration, environment record, gate results, raw timings, signatures, pcap and report under one run_id.

*Evidence: results/runs/<run_id>/*

**Trade-offs we accept**
One CPU core and no network make runs repeatable, but absolute timings are not production numbers: compare ratios. MTL gains depend on zone size and ladder reuse.

**Extension path**
Swap in mentor-provided algorithm builds, add ML-DSA-MTL (draft-kaizer-dnsop-ml-dsa-mtl-dnssec), and test multi-core runs and larger zones.

# Data Flow Diagram

**Level-1 DFD of one benchmark run: from the configuration to the report.**

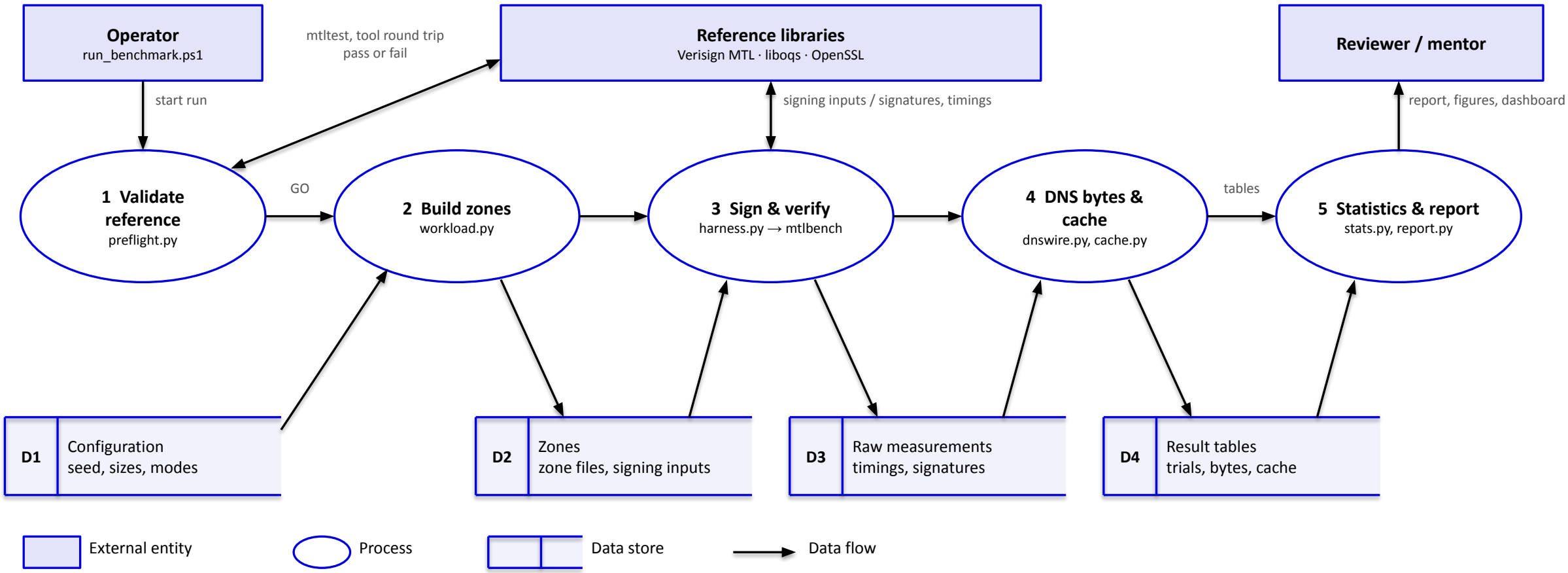



Figure: Data Flow Diagram (DFD)

# Deployment Timeline

**Milestones, and how the platform will be progressed towards usable deployment.**



**Sep 30, 2026**
**Scope and design**
PS-036 analysis, RFC and draft review, repository created
**Done**

**Oct 6-10, 2026**
**Develop MVP prototype-1**
Pinned image, go/no-go gate, C harness, zone builder, DNS and cache models, report, dashboard
**Done**

**Oct 10-13, 2026**
**Quick demo run**
3 trials × 3 zones × 5 modes; gate GO; every correctness check passed
**Done**

**Oct 14, 2026**
**Submission**
Report; private GitHub repository transferred to aiori-hackathon; mentor demo
**Done**

**After Oct 16-31**
**Mentor sessions**
Confirming the MTL implementation and incorporating the solution in the code and the live demo.
**Planned**

## Deployment methodology

- **One artifact.** Everything runs from one Docker image built from the repository; the host needs only Docker and git.
- **One command.** A single script builds the image and runs the benchmark (configuration and CPU cores selectable); a second runs the live validation on the finished run.
- **Self-describing runs.** Each run keeps its configuration, environment record, gate results, raw data, signatures, packet captures and report, so every number can be traced and re-run.
- **Tested.** pytest covers workloads, DNS wire format, cache and formulas on the host; the harness and library gates run inside the container.

# Bibliography I


[1] R. Arends, R. Austein, M. Larson, D. Massey, S. Rose. “DNS Security Introduction and Requirements”; “Resource Records for the DNS Security Extensions”; “Protocol Modifications for the DNS Security Extensions”. In: *RFC 4033, 4034, 4035* (2005).

[2] B. Laurie, G. Sisson, R. Arends, D. Blacka. “DNS Security (DNSSEC) Hashed Authenticated Denial of Existence”. In: *RFC 5155* (2008). W. Hardaker, V. Dukhovni. “Guidance for NSEC3 Parameter Settings”. In: *RFC 9276* (2022).

[3] J. Damas, M. Graff, P. Vixie. “Extension Mechanisms for DNS (EDNS(0))”. In: *RFC 6891* (2013). J. Dickinson, S. Dickinson, R. Bellis, A. Mankin, D. Wessels. “DNS Transport over TCP – Implementation Requirements”. In: *RFC 7766* (2016).

[4] National Institute of Standards and Technology. “Stateless Hash-Based Digital Signature Standard”. In: *FIPS 205* (2024).

[5] J. Harvey et al. “Merkle Tree Ladder (MTL) Mode Signatures”. In: *Internet-Draft draft-harvey-cfrg-mtl-mode-09*, IRTF CFRG (work in progress).

[6] A. Fregly et al. “Stateless Hash-Based Signatures in Merkle Tree Ladder Mode (SLH-DSA-MTL) for DNSSEC”. In: *Internet-Draft draft-fregly-dnsop-slh-dsa-mtl-dnssec-05*, IETF DNSOP (work in progress).

# Bibliography II

[7] A. Fregly, J. Harvey, B. S. Kaliski Jr., S. Sheth. “Merkle Tree Ladder Mode: Reducing the Size Impact of NIST PQC Signature Algorithms in Practice”. In: *Topics in Cryptology – CT-RSA 2023*, LNCS 13871, Springer (2023).

[8] Verisign. “MTL Mode reference implementation”. github.com/verisign/MTL, commit 7cef019.

[9] Open Quantum Safe. “liboqs”, version 0.16.0. github.com/open-quantum-safe/liboqs. OpenSSL Project. “OpenSSL”, version 3.6.0. openssl.org.

[10] J. Jansen. “Use of SHA-2 Algorithms with RSA in DNSKEY and RRSIG Resource Records for DNSSEC”. In: *RFC 5702* (2009). P. Hoffman, W. Wijngaards. “Elliptic Curve Digital Signature Algorithm (DSA) for DNSSEC”. In: *RFC 6605* (2012). O. Surý, R. Edmonds. “Edwards-Curve Digital Security Algorithm (EdDSA) for DNSSEC”. In: *RFC 8080* (2017).

[11] DNS Flag Day. “DNS Flag Day 2020”. dnsflagday.net/2020.

Thank You for Your Attention!